



Pontificia Universidad Católica de Chile
Departamento de Ciencias de la Computación

Diplomado en Inteligencia Artificial

Clase 20

ELMo, BERT, GPT, ChatGPT

Carlos Aspillaga



IA Lab

CENIA
CENTRO NACIONAL DE INTELIGENCIA ARTIFICIAL

Contenidos de la clase de hoy

1

Introducción

¿Por qué tantos modelos?

2

ELMo

Modelo
Aplicaciones

3

BERT

Modelo
Aplicaciones

4

GPT

Modelos GPT 1, 2 y 3, chatGPT y GPT-4
Aplicaciones

5

Herramientas Concretas

Introducción
Trabajo práctico

Contenidos de la clase de hoy

1

Introducción

¿Por qué tantos modelos?

2

ELMo

Modelo

Aplicaciones

3

BERT

Modelo

Aplicaciones

4

GPT

Modelos GPT 1, 2 y 3, chatGPT y GPT-4

Aplicaciones

5

Herramientas Concretas

Introducción

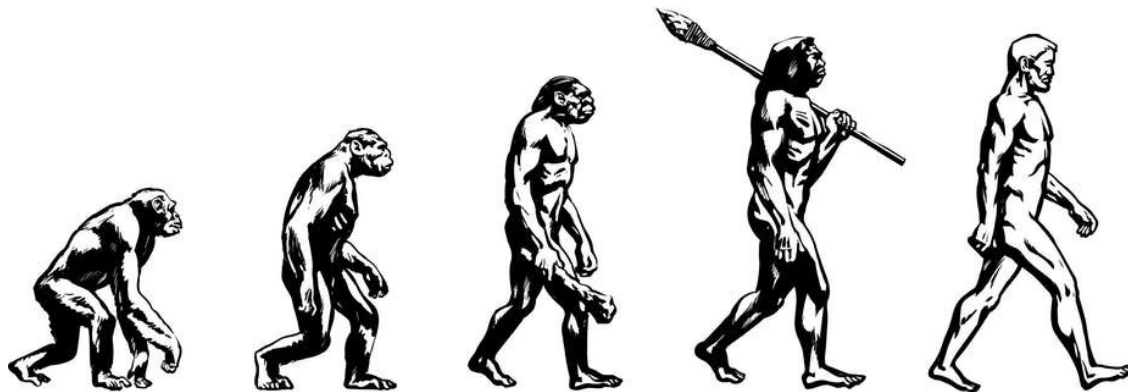
Trabajo práctico

Fuentes consultadas:

- Deeplearn.ai
- Pablo Messina
- Papers originales de cada modelo

Introducción

¿Por qué tantos modelos?



- Mejores arquitecturas
- Mejores estrategias de entrenamiento
- Más o mejores datos
- Mayor tamaño o nivel de cómputo

1

El NLP evoluciona (muy rápido)

Introducción

¿Por qué tantos modelos?



1

El NLP evoluciona (muy rápido)

2

Distintos modelos resuelven distintos problemas

Introducción

Modelos de esta clase

Diseñado para
“Entender un texto”

BERT
ELMo



Diseñados para
“Generar texto”

GPT-1



GPT-2



GPT-3



Introducción

Modelos de esta clase

Texto → Clasificación

Diseñado para
“Entender un texto”

BERT
ELMo



Texto → Continuación del texto

Diseñados para
“Generar texto”

GPT-1 

GPT-2 

GPT-3 

Introducción

Modelos de esta clase

idea revolucionaria / entrenamiento revolucionario.
+ Mucho cómputo* *(para ese tiempo)

BERT
ELMo



Muuuuchos parámetros
Muuuuchos datos
Muuuucho cómputo

GPT-1 

GPT-2 

GPT-3 

Contenidos de la clase de hoy

1

Introducción

¿Por qué tantos modelos?

2

ELMo

Modelo

Aplicaciones

3

BERT

Modelo

Aplicaciones

4

GPT

Modelos GPT 1, 2 y 3, chatGPT y GPT-4

Aplicaciones

5

Herramientas Concretas

Introducción

Trabajo práctico

Deep Contextualized Word Representations (2018)

Deep contextualized word representations

Matthew E. Peters[†], Mark Neumann[†], Mohit Iyyer[†], Matt Gardner[†],
`{matthewp, markn, mohiti, mattg}@allenai.org`

Christopher Clark*, Kenton Lee*, Luke Zettlemoyer^{†*}
`{csquared, kentonl, lsz}@cs.washington.edu`

[†]Allen Institute for Artificial Intelligence

*Paul G. Allen School of Computer Science & Engineering, University of Washington

Deep Contextualized Word Representations (2018)



Deep Contextualized Word Representations

Contexto previo:

- Word embeddings tradicionales hasta el momento: Word2vec, GloVe, FastText

Limitaciones:

- Embeddings tradicionales son estáticos: el vector de la palabra es siempre el mismo, no depende del contexto que la rodea.
- Por lo anterior, no son tan robustos manejando polisemia:

- **arm**

I have an ant bite on my **arm**.
It's important to **arm** yourself with a solid education.

- **fall**

I love cool, crisp **fall** weather.
Don't **fall** on your way to the gym.

- **clip**

I enjoyed watching a **clip** from that video.
My mom is going to **clip** my hair.
The boat is moving at a fairly fast **clip**.

- **drop**

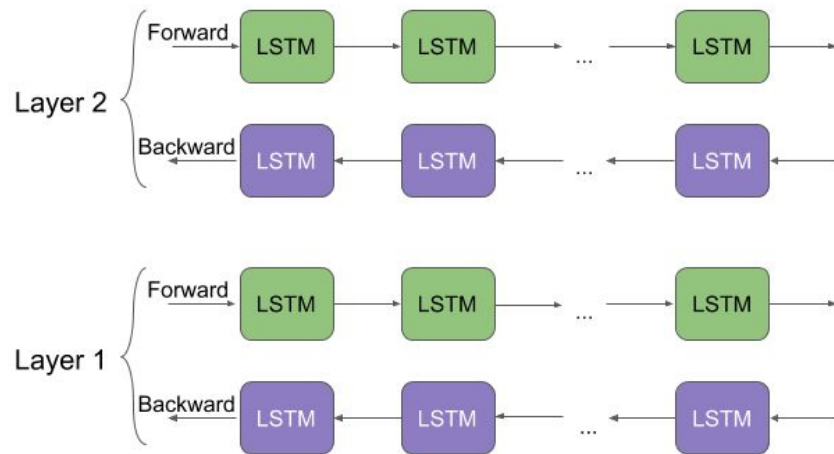
I hope I don't **drop** my books.
I enjoyed every last **drop** of my milkshake.

Deep Contextualized Word Representations

- Deep: Redes neuronales profundas
- Contextualized: word vector = función(palabra|contexto)
- Word: palabras como unidad semántica fundamental
- Representations: Embeddings (vectores distribuidos en un espacio continuo)

Objetivo: obtener word embeddings contextualizados (dependientes del contexto)

Deep Contextualized Word Representations



Deep Contextualized Word Representations:

Resumen del modelo

- 1) Obtener un embedding inicial por palabra a partir de sus caracteres:
Character-level embedding → Word-level embedding (inicial, sin info. de contexto)
- 2) Se usan varias capas de LSTMs bidireccionales (BiLSTM) para procesar un texto, obteniéndose un vector contextualizado por cada palabra en cada capa.
 - forward Language Model (fLM): predecir palabra **siguiente** dadas las **anteriores**
 - backward Language Model (bLM): predecir palabra **anterior** dadas las **siguientes**
- 3) Luego, el biLM pre-entrenado se usa para obtener un vector contextualizado final por palabra calculando un promedio ponderado del embedding inicial y los vectores contextualizados de cada capa. Esto se conoce como Embedding from Language Models (ELMo).
- 4) Se aprende un ELMo diferente por tarea (task-specific ELMo)

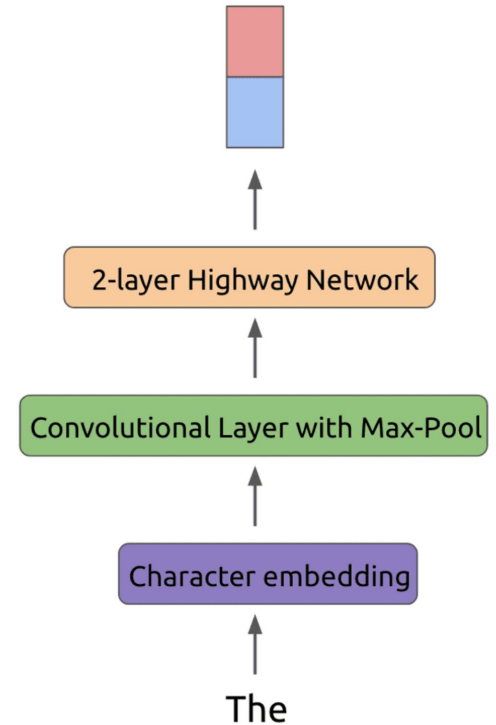
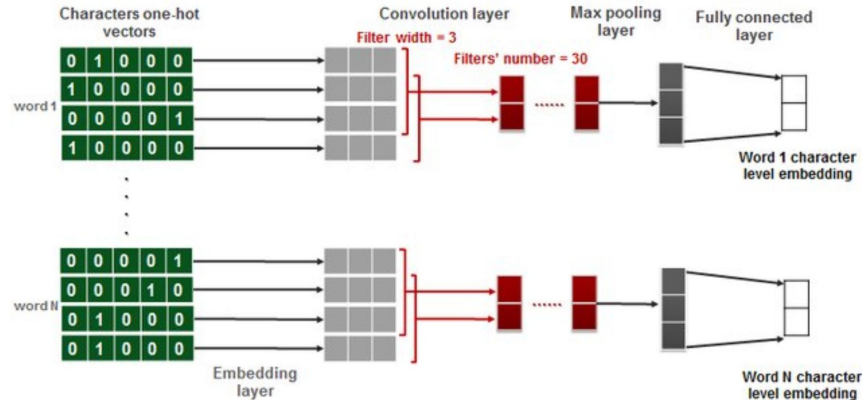


Deep Contextualized Word Representations:

1) Character-level embeddings

Ventajas:

- Generalizar a palabras nuevas (out-of-vocabulary words)
- Capturar detalles a nivel de caracteres (conjugaciones, plural, etc.)
- Evitamos hacer stemming, lemmatization, etc.



Deep Contextualized Word Representations:

2) Bidirectional Language Model

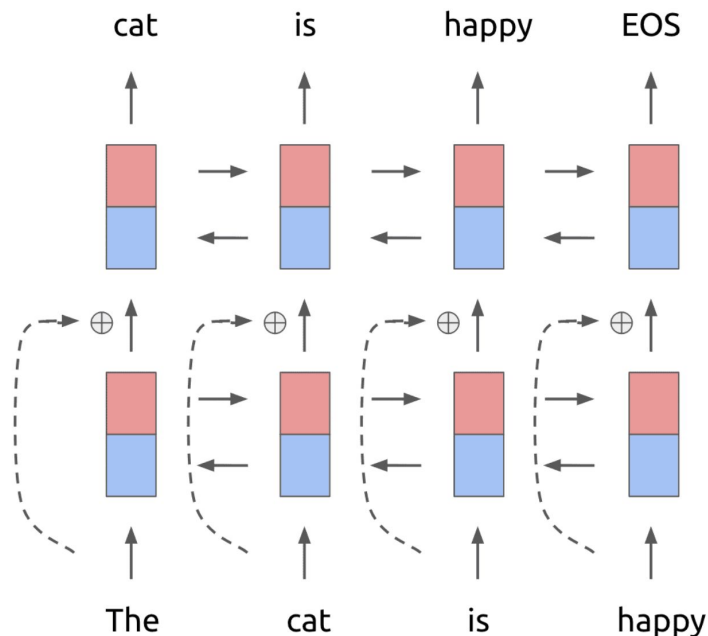
- 1) Forward Language Model: predecir siguiente palabra dadas las anteriores

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_1, t_2, \dots, t_{k-1}).$$

- 2) Backward Language Model: predecir palabra anterior dadas las siguientes

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_{k+1}, t_{k+2}, \dots, t_N)$$

$$\sum_{k=1}^N (\log p(t_k | t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) + \log p(t_k | t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s))$$

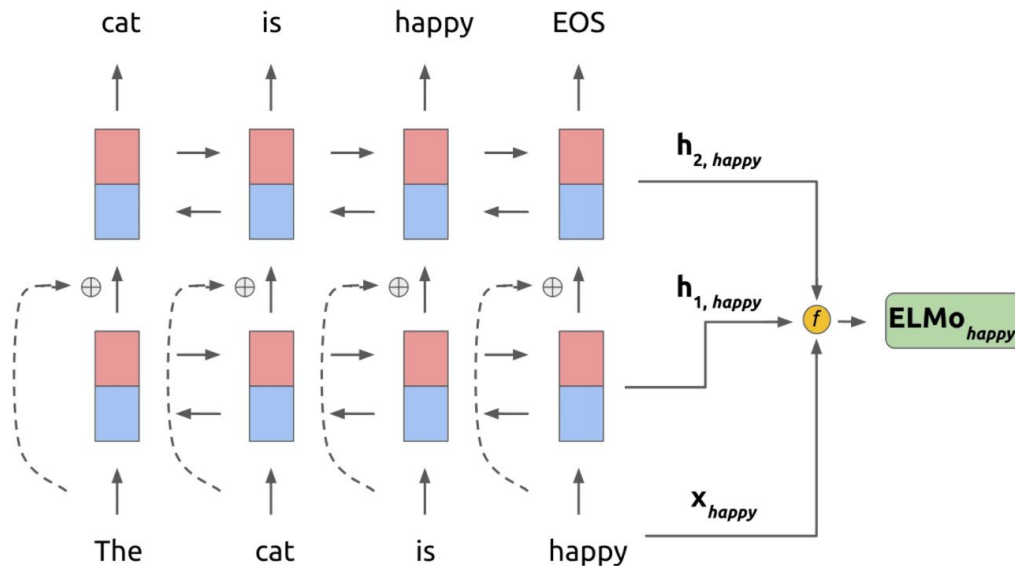




Deep Contextualized Word Representations:

3) Embeddings from Language Models (ELMo)

1. Usa el biLM preentrenado para obtener word embeddings contextualizados.
2. El embedding final de cada palabra es una combinación lineal de las representaciones de cada capa.
3. Los pesos de la comb. lineal. se aprenden (task specific). Pueden haber múltiples combinaciones lineales distintas.



$$ELMo_k^{task} = \gamma_k \cdot (s_0^{task} \cdot x_k + s_1^{task} \cdot h_{1,k} + s_2^{task} \cdot h_{2,k})$$

Deep Contextualized Word Representations:

4) Entrenamiento y Evaluación

- Datos de entrenamiento: One Billion Word Benchmark.
- Luego se evaluaron en varios datasets, mejorando el estado del arte en todos!

TASK	PREVIOUS SOTA		OUR BASELINE	ELMo + BASELINE	INCREASE (ABSOLUTE/ RELATIVE)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

Contenidos de la clase de hoy

1 **Introducción**
¿Por qué tantos modelos?

2 **ELMo**
Modelo
Aplicaciones

3 **BERT**
Modelo
Aplicaciones

4 **GPT**
Modelos GPT 1, 2 y 3, chatGPT y GPT-4
Aplicaciones

5 **Herramientas Concretas**
Introducción
Trabajo práctico

BERT: Bidirectional Encoder Representations from Transformers (2019)



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language

`{jacobdevlin, mingweichang, kentonl, kristout}@google.com`

BERT: Bidirectional Encoder Representations from Transformers (2019)



Learning new things is fun!

Prompting is a powerful developer tool.

lollipop

l-o-l-l-i-p-o-p

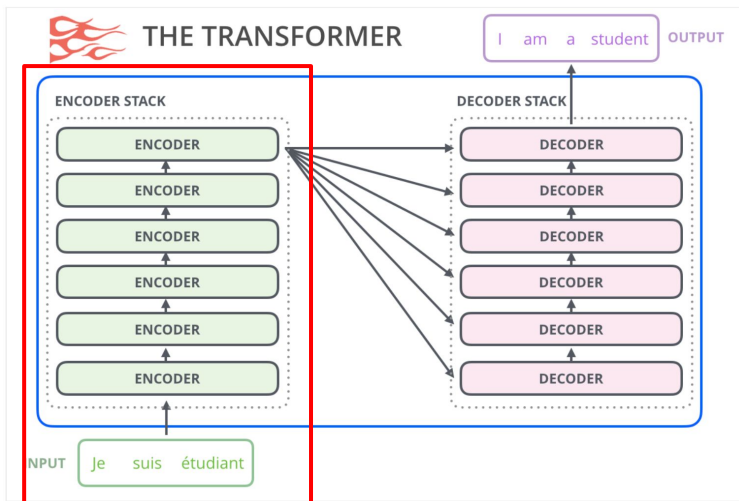
Tokenización mediante “pedazos” de palabras

Permite manejar palabras nuevas

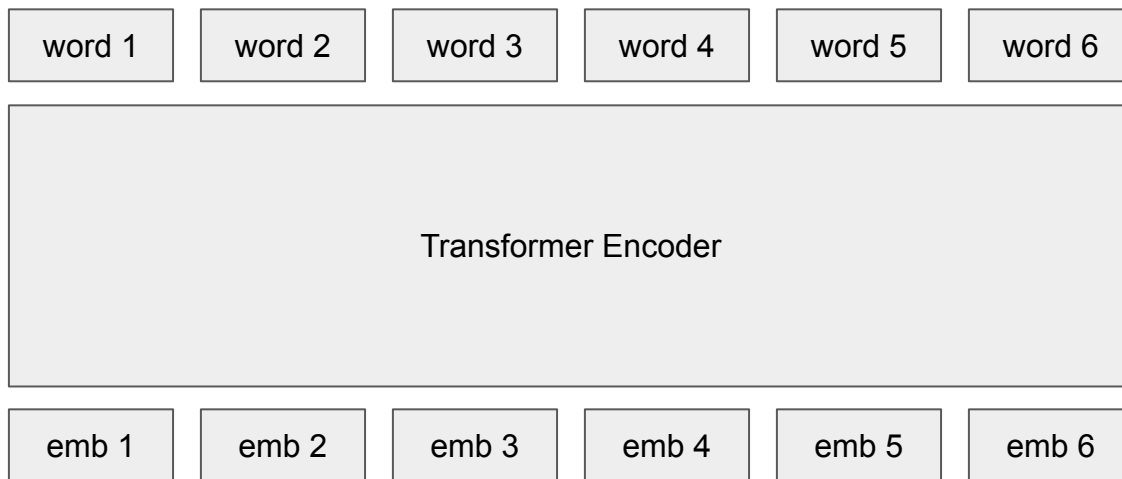
BERT: Bidirectional Encoder Representations from Transformers



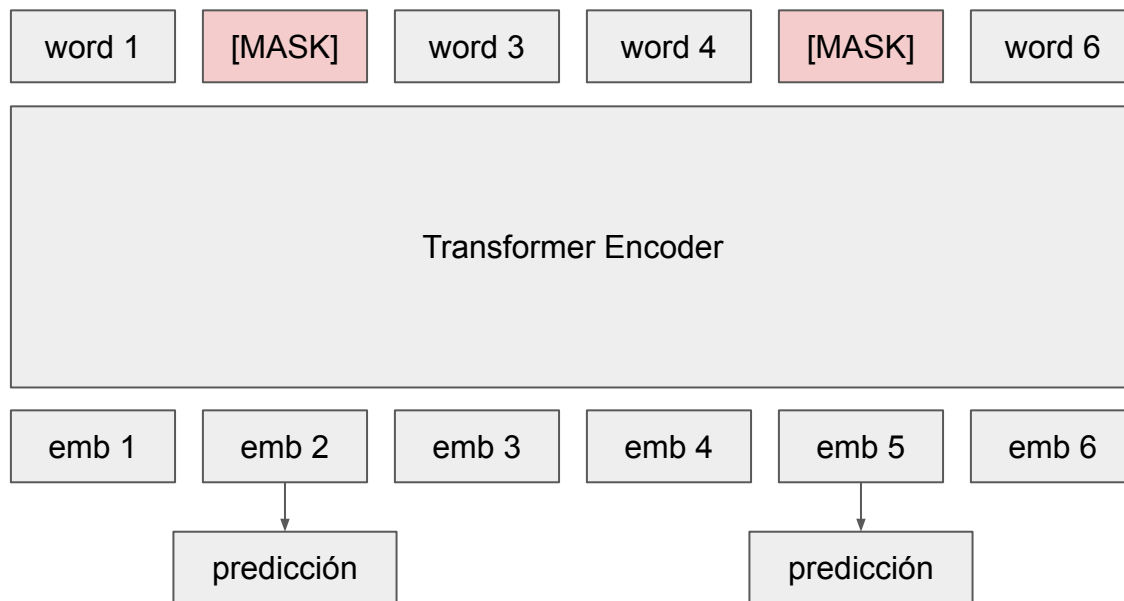
- BERT usa el Encoder del Transformer.



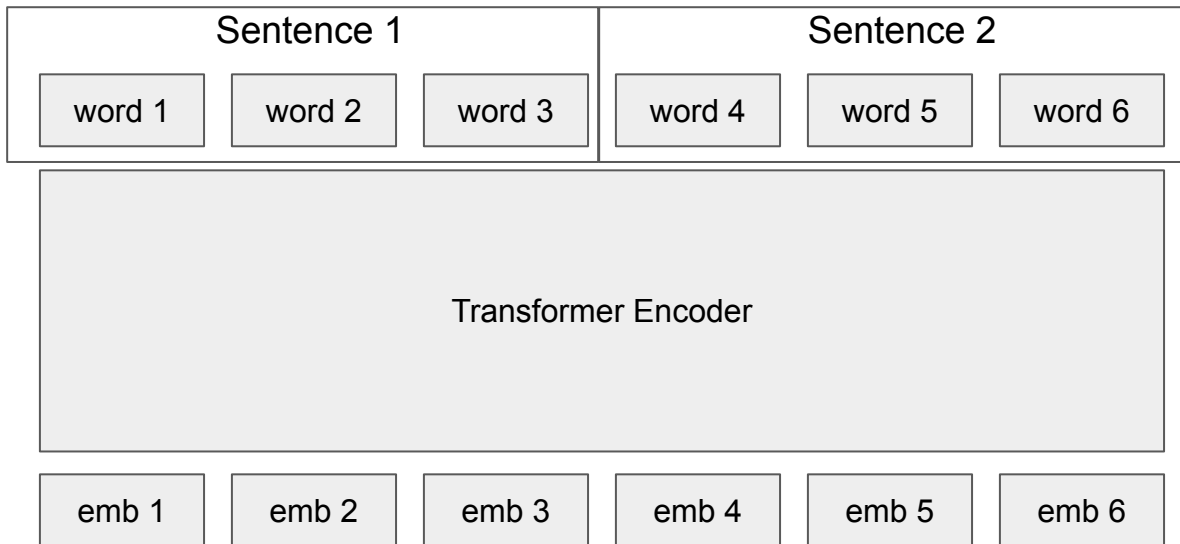
BERT: Bidirectional Encoder Representations from Transformers



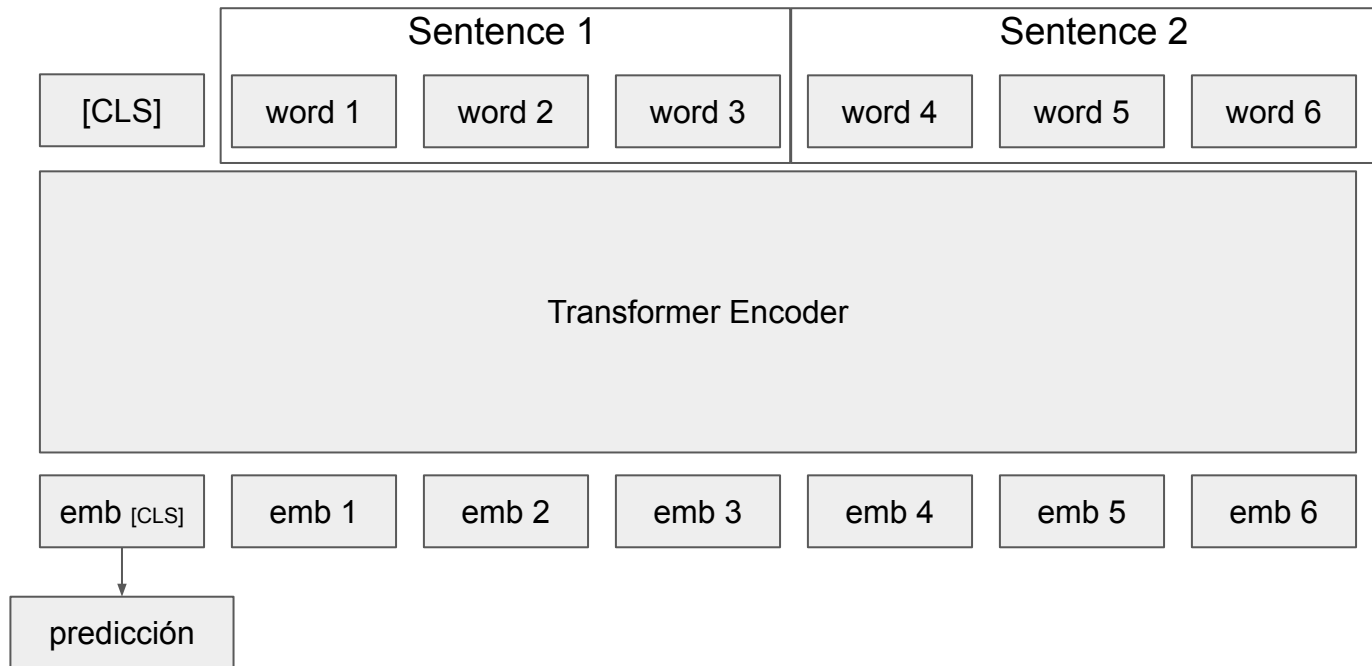
BERT: Bidirectional Encoder Representations from Transformers



BERT: Bidirectional Encoder Representations from Transformers



BERT: Bidirectional Encoder Representations from Transformers



BERT: Bidirectional Encoder Representations from Transformers

1) Entradas

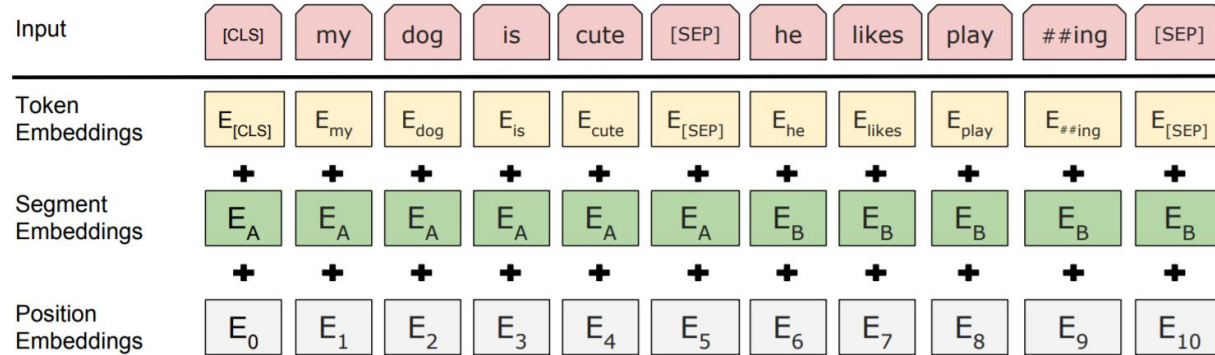
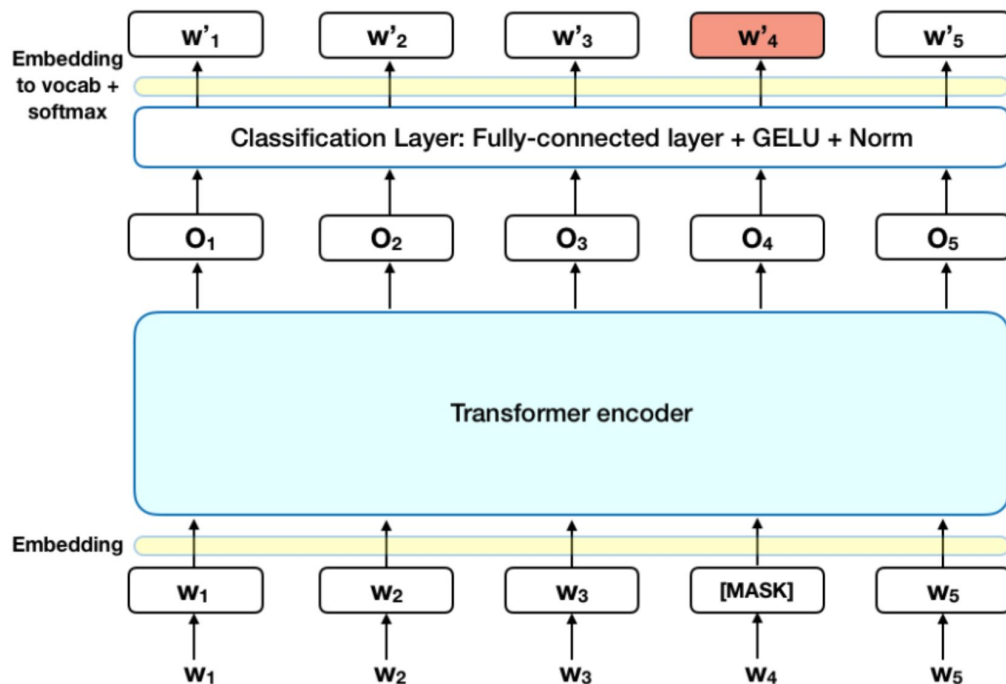


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

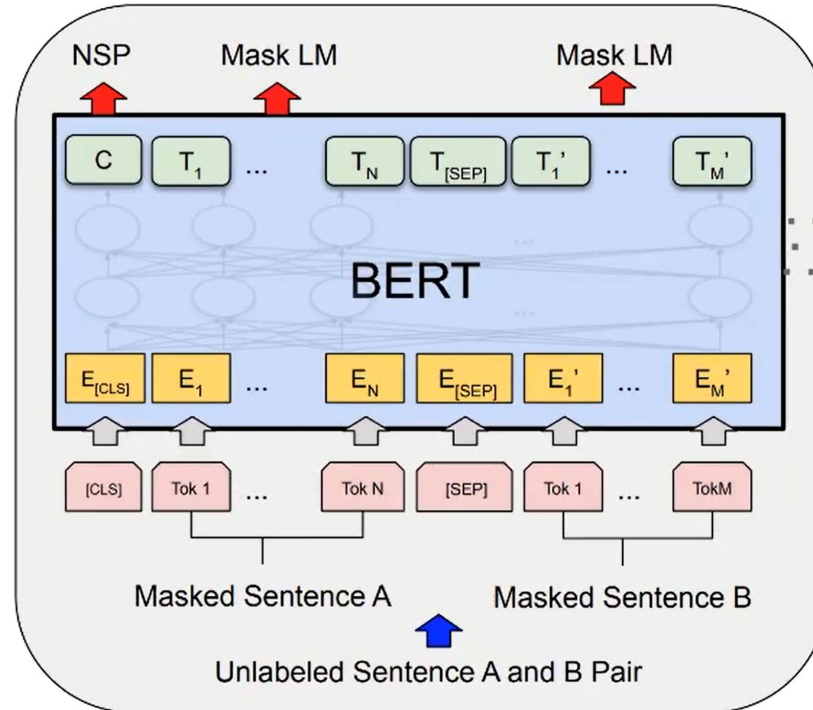
BERT: Bidirectional Encoder Representations from Transformers

2) BERT propiamente tal



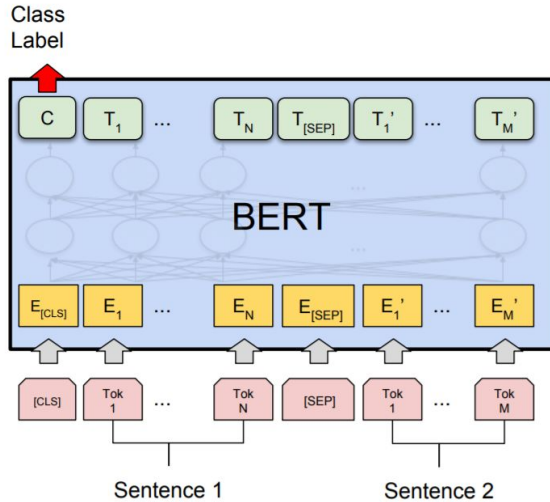
BERT: Bidirectional Encoder Representations from Transformers

3) Pre-entrenamiento: Masked Language Model + Next Sentence Prediction

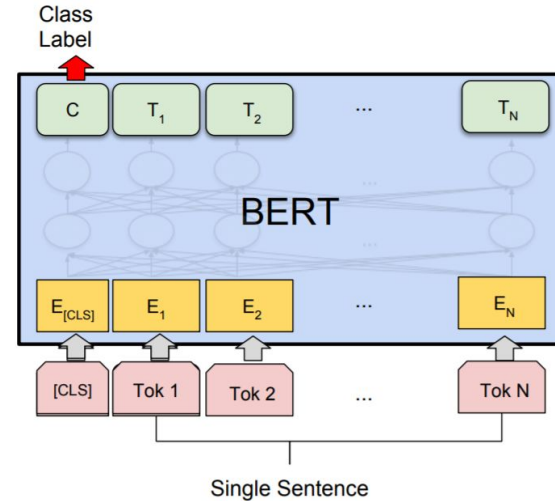


BERT: Bidirectional Encoder Representations from Transformers

4) Fine-tuning en tareas específicas



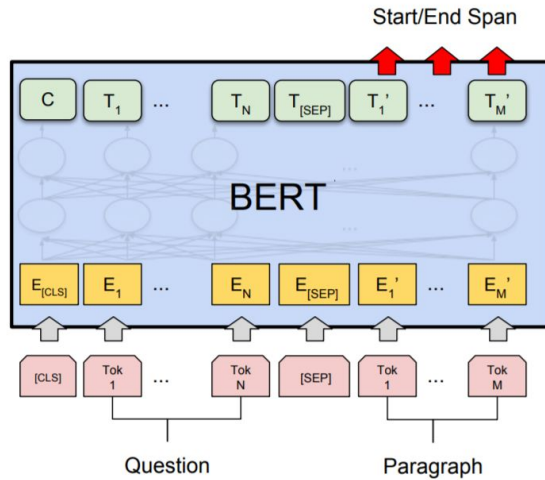
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



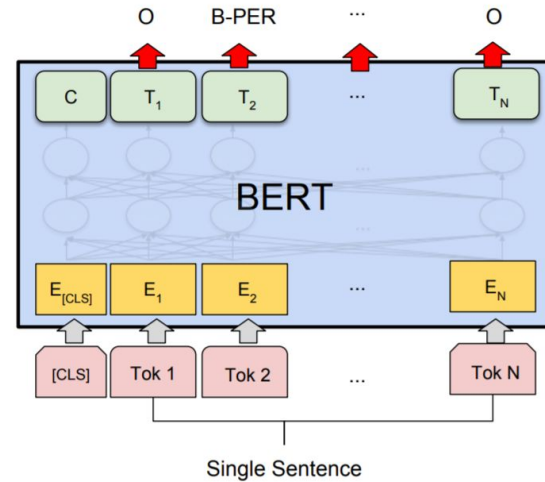
(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT: Bidirectional Encoder Representations from Transformers

4) Fine-tuning en tareas específicas



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT: Bidirectional Encoder Representations from Transformers

5) Algunos resultados



System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

BERT: Bidirectional Encoder Representations from Transformers

6) Detalles técnicos



- BERT base : 110 millones de parámetros
- BERT large: 340 millones de parámetros
- Entradas: word piece embeddings (30 mil tokens)
- Datasets de pre-entrenamiento:
 - BooksCorpus: 800 millones de palabras.
 - English Wikipedia: 2500 millones de palabras.

Evoluciones de BERT

RoBERTa

Mismo tamaño de modelo

- + Mejor búsqueda de hiper-parámetros
- + Más datos
- + Más cómputo

= Mejor rendimiento

Evoluciones de BERT

BETO y BERTÍN

Mismo modelo

Datos en español

Cómputo similar

= Modelo en español

Evoluciones de BERT

BETO y BERTÍN

Models 207

🔍 dccuchile



Full-text search

↕ Sort: Trending

🔍 dccuchile/albert-base-spanish

Updated Apr 28, 2022 • ⬇ 78 • ❤ 4

🔍 dccuchile/albert-tiny-spanish

Updated Apr 28, 2022 • ⬇ 51 • ❤ 3

🔍 dccuchile/bert-base-spanish-wwm-cased

📄 Fill-Mask • Updated Jan 17 • ⬇ 42.5k • ❤ 52

🔍 dccuchile/bert-base-spanish-wwm-uncased

📄 Fill-Mask • Updated Jan 17 • ⬇ 243k • ❤ 61

🔍 dccuchile/patana-chilean-spanish-bert

📄 Fill-Mask • Updated Mar 5 • ⬇ 21 • ❤ 5

🔍 dccuchile/albert-base-spanish-finetuned-mldoc

📄 Text Classification • Updated Jan 10, 2022 • ⬇ 18

🔍 dccuchile/albert-base-spanish-finetuned-ner

📄 Token Classification • Updated Dec 28, 2021 • ⬇ 20

🔍 dccuchile/albert-base-spanish-finetuned-pawsx

📄 Text Classification • Updated Jan 1, 2022 • ⬇ 13

🔍 dccuchile/albert-base-spanish-finetuned-pos

📄 Token Classification • Updated Dec 17, 2021 • ⬇ 9

🔍 dccuchile/albert-base-spanish-finetuned-qa-mlqa

📄 Question Answering • Updated Jan 18, 2022 • ⬇ 32 • ❤ 3

🔍 dccuchile/albert-base-spanish-finetuned-xnli

📄 Text Classification • Updated Dec 8, 2021 • ⬇ 9

🔍 dccuchile/albert-large-spanish-finetuned-mldoc

📄 Text Classification • Updated Jan 11, 2022 • ⬇ 11

Evoluciones de BERT

BETO y BERTÍN

Models 27

bertin-project

Full-text search

Sort: Trending

 bertin-project/bertin-base-gaussian-exp-512seqlen
Fill-Mask • Updated Sep 23, 2021 • ↓ 34 • ♥ 1

 bertin-project/bertin-base-gaussian
Fill-Mask • Updated Sep 23, 2021 • ↓ 8

 bertin-project/bertin-base-ner-conll2002-es
Token Classification • Updated Apr 4, 2023 • ↓ 21 • ♥ 1

 bertin-project/bertin-base-paws-x-es
Text Classification • Updated Mar 21, 2023 • ↓ 13 • ♥ 1

 bertin-project/bertin-base-pos-conll2002-es
Token Classification • Updated Jun 30, 2023 • ↓ 28 • ♥ 2

 bertin-project/bertin-base-random-exp-512seqlen
Fill-Mask • Updated Sep 23, 2021 • ↓ 19

 bertin-project/bertin-base-random
Fill-Mask • Updated Sep 23, 2021 • ↓ 29

 bertin-project/bertin-base-stepwise-exp-512seqlen
Fill-Mask • Updated Sep 23, 2021 • ↓ 19

 bertin-project/bertin-base-stepwise
Fill-Mask • Updated Sep 23, 2021 • ↓ 22

 bertin-project/bertin-base-xnli-es
Text Classification • Updated Mar 26, 2023 • ↓ 14 • ♥ 1

 bertin-project/bertin-roberta-base-spanish
Fill-Mask • Updated Jul 17 • ↓ 37k • ♥ 35

 bertin-project/bertin-gpt-j-6B
Text Generation • Updated 5 days ago • ↓ 3.37k • ♥ 17

 bertin-project/bertin-gpt-j-6B-infolibros
Text Generation • Updated Dec 12, 2022 • ↓ 50

 bertin-project/corin
Text Generation • Updated Jan 10, 2023 • ↓ 14 • ♥ 2

Contenidos de la clase de hoy

1

Introducción

¿Por qué tantos modelos?

2

ELMo

Modelo
Aplicaciones

3

BERT

Modelo
Aplicaciones

4

GPT

Modelos GPT 1, 2 y 3, chatGPT y GPT-4
Aplicaciones

5

Herramientas Concretas

Introducción
Trabajo práctico

GPT (2018)

Improving Language Understanding by Generative Pre-Training

Alec Radford
OpenAI
alec@openai.com

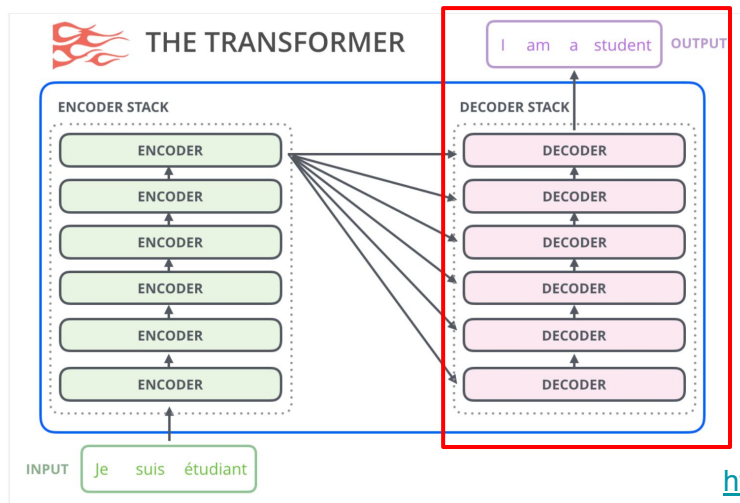
Karthik Narasimhan
OpenAI
karthikn@openai.com

Tim Salimans
OpenAI
tim@openai.com

Ilya Sutskever
OpenAI
ilyasu@openai.com

GPT

- Sólo se utiliza el **Decoder** del Transformer.
- Pensado para **generación de texto**.



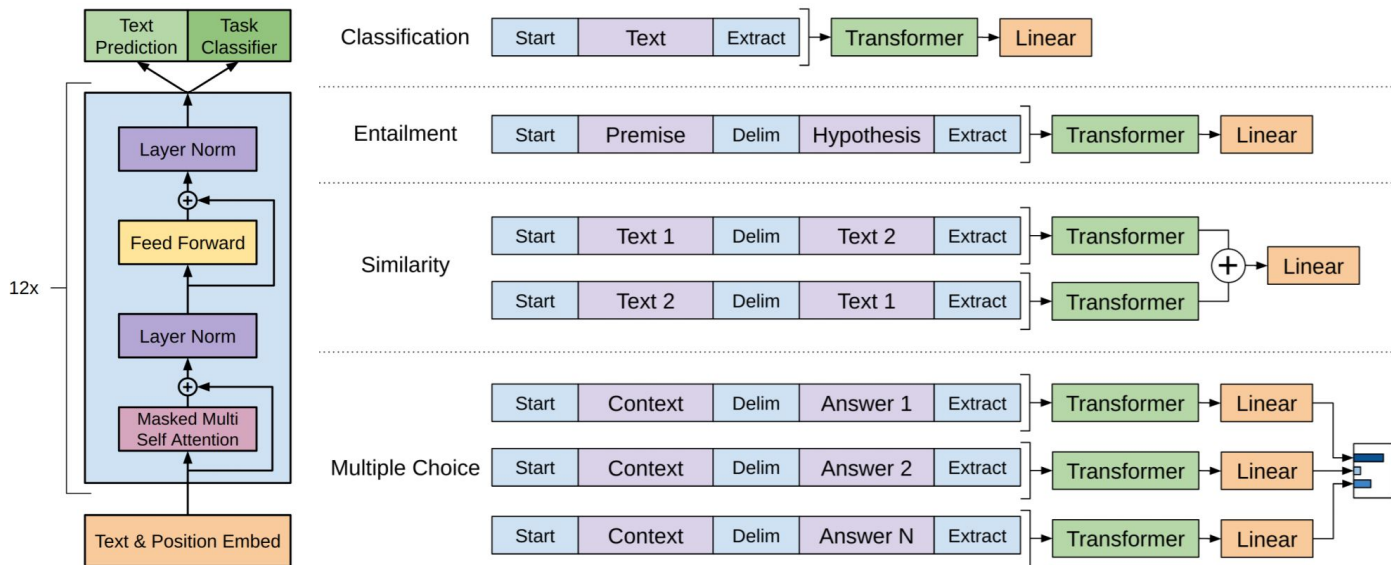
GPT: Pre-entrenamiento no-supervisado / autosupervisado

Tarea: dado un corpus de palabras, aprender un modelo de lenguaje que calcule la probabilidad de la siguiente palabra dada las k palabras anteriores.

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

Dataset usado: BookCorpus, 7000 libros de varios géneros, con textos largos (dependencias largas entre palabras).

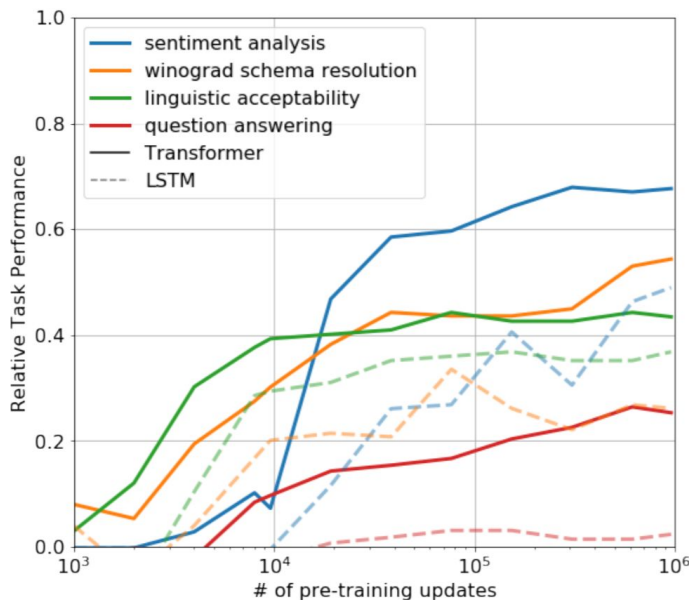
GPT: Fine-tuning supervisado



GPT: Resultados notables

- GPT fue evaluado en 12 datasets, logrando mejorar el estado del arte en 9.
- **Zero-shot behaviors** (generalización sin fine-tuning, sólo con el pre-entrenamiento no-supervisado):

El modelo es capaz de resolver tareas para las cuales no fue entrenado sin necesidad de hacer fine-tuning.



GPT 2 (2019)

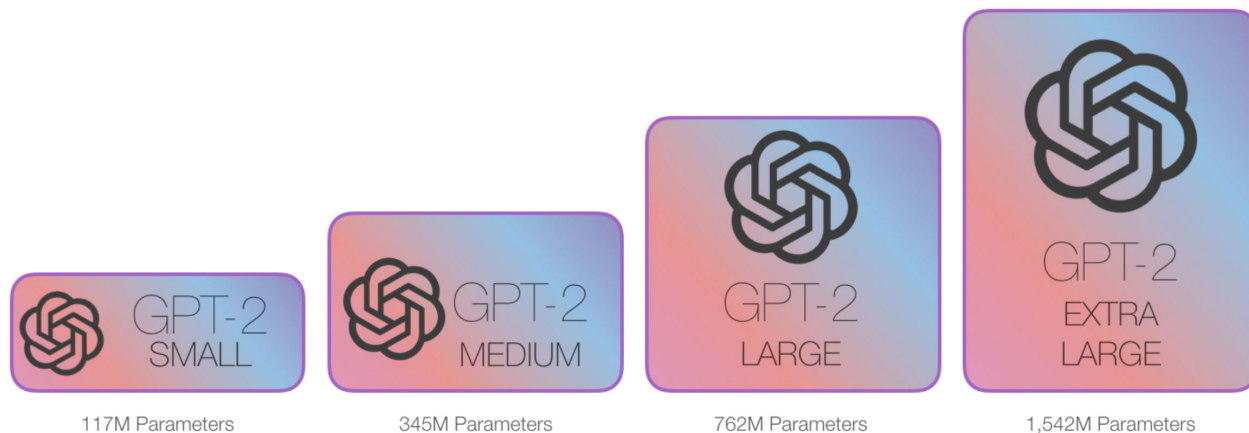
Language Models are Unsupervised Multitask Learners

Alec Radford^{* 1} Jeffrey Wu^{* 1} Rewon Child¹ David Luan¹ Dario Amodei^{ 1} Ilya Sutskever^{** 1}**

^{*, **}Equal contribution ¹OpenAI, San Francisco, California, United States. Correspondence to: Alec Radford <alec@openai.com>.

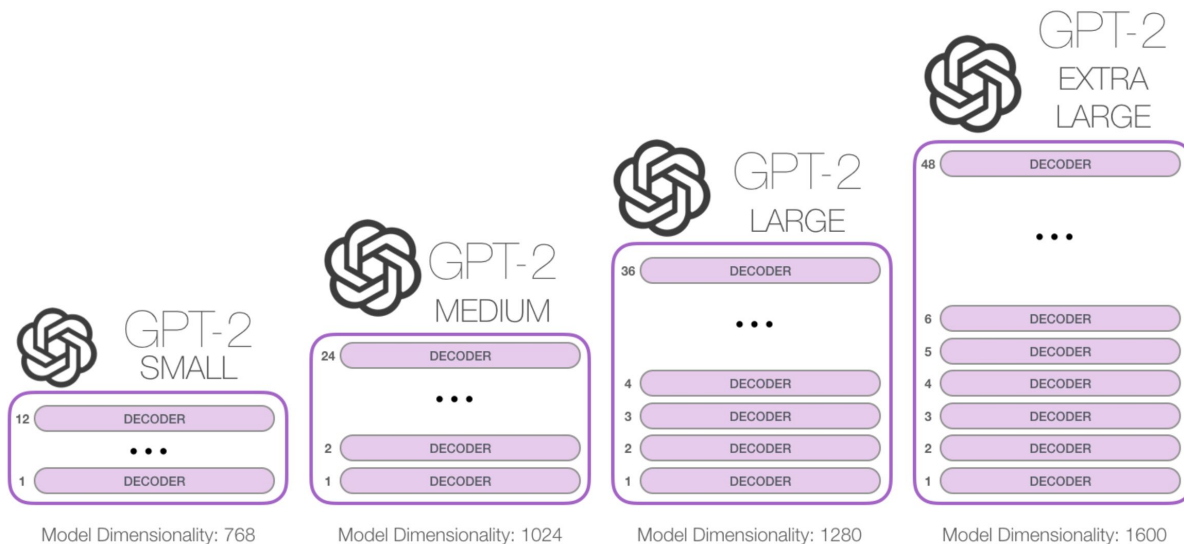
GPT 2: Resumen del modelo

- Salvo detalles menores, prácticamente la misma arquitectura que GPT
- Principal diferencia: el tamaño.



GPT 2: Resumen del modelo

- Salvo detalles menores, prácticamente la misma arquitectura que GPT
- Principal diferencia: el tamaño.



GPT 2: Entrenamiento y Evaluación

Dataset de entrenamiento

WebText: consistente en 45 millones de links publicados en Reddit (con un nivel mínimo de reputación para asegurar calidad). Luego de deduplicar documentos y algunos filtros, quedaron 8 millones de documentos (40 GB de texto).

Datasets de evaluación y resultados

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

Table 3. Zero-shot results on many datasets. No training or fine-tuning was performed for any of these results. PTB and WikiText-2 results are from (Gong et al., 2018). CBT results are from (Bajgar et al., 2016). LAMBADA accuracy result is from (Hoang et al., 2018) and LAMBADA perplexity result is from (Grave et al., 2016). Other results are from (Dai et al., 2019).

GPT 2: Resultados más notables

- GPT 2 mejoró el estado del arte en 7 de los 8 datasets.
- **Zero-shot generalization**: GPT 2 logró dichos resultados sin fine-tuning, sólo con el pre-entrenamiento en WebText
- Lo anterior sugiere que un modelo de lenguaje con muchos parámetros y entrenado en un corpus suficientemente grande y variado puede aprender una gran variedad de tareas, permitiendo una fácil generalización a nuevos datasets.

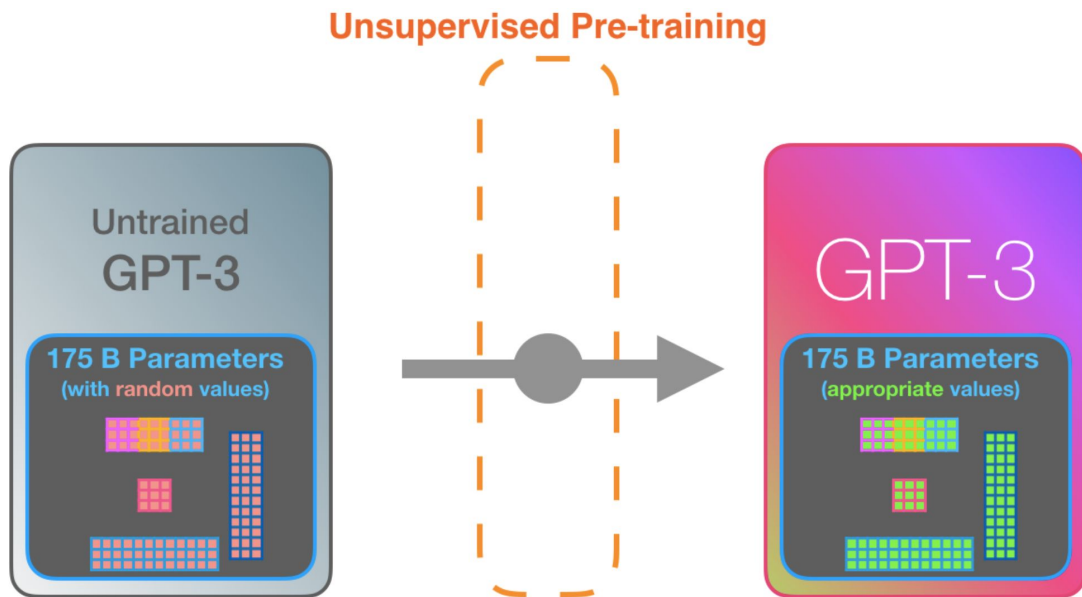
GPT 3 (2020)

Language Models are Few-Shot Learners				
Tom B. Brown*		Benjamin Mann*	Nick Ryder*	Melanie Subbiah*
Jared Kaplan†	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	
OpenAI				

Link recomendado: <http://jalammar.github.io/how-gpt3-works-visualizations-animations/>

GPT 3: Resumen del modelo

Misma historia: igual que GPT 2 pero **MUCHO** más grande.



GPT 3: Entrenamiento

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

GPT 3: Zero-shot, One-shot, Few-shot y Fine-tuning

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.

```
1 sea otter => loutre de mer ← example #1
↓
gradient update
↓
1 peppermint => menthe poivrée ← example #2
↓
gradient update
↓
...
↓
1 plush giraffe => girafe peluche ← example #N
↓
gradient update
↓
1 cheese => ..... ← prompt
```

Más ejemplos en:
<https://platform.openai.com/examples/>

GPT 3: Resultados notables

- GPT 3 alcanza un desempeño con la técnica few-shot comparable a los mejores modelos que usan fine-tuning, en tareas como:
 - translation, question-answering, cloze tasks
 - unscrambling words,
 - using novel word in a sentence
 - 3-digit arithmetic
- GPT 3 es capaz de generar artículos nuevos que son casi indistinguibles de un artículo redactado por un humano.

	Mean accuracy	95% Confidence Interval (low, hi)	<i>t</i> compared to control (<i>p</i> -value)	"I don't know" assignments
Control (deliberately bad model)	86%	83%–90%	-	3.6 %
GPT-3 Small	76%	72%–80%	3.9 (2e-4)	4.9%
GPT-3 Medium	61%	58%–65%	10.3 (7e-21)	6.0%
GPT-3 Large	68%	64%–72%	7.3 (3e-11)	8.7%
GPT-3 XL	62%	59%–65%	10.7 (1e-19)	7.5%
GPT-3 2.7B	62%	58%–65%	10.4 (5e-19)	7.1%
GPT-3 6.7B	60%	56%–63%	11.2 (3e-21)	6.2%
GPT-3 13B	55%	52%–58%	15.3 (1e-32)	7.1%
GPT-3 175B	52%	49%–54%	16.9 (1e-34)	7.8%

Table 3.11: Human accuracy in identifying whether short (~200 word) news articles are model generated. We find that human accuracy (measured by the ratio of correct assignments to non-neutral assignments) ranges from 86% on the control model to 52% on GPT-3 175B. This table compares mean accuracy between five different models, and shows the results of a two-sample T-Test for the difference in mean accuracy between each model and the control model (an unconditional GPT-3 Small model with increased output randomness).

chatGPT (instructGPT):

2203.02155v1 [cs.CL] 4 Mar 2022

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell[†]

Peter Welinder

Paul Christiano*[†]

Jan Leike*

Ryan Lowe*

OpenAI

Abstract

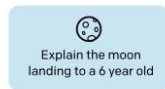
Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on

chatGPT (instructGPT):

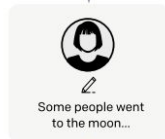
Step 1

**Collect demonstration data,
and train a supervised policy.**

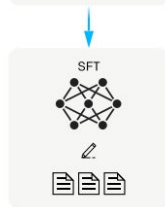
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

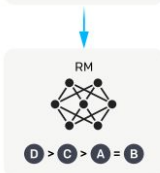
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



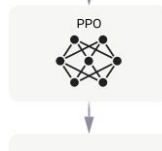
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.

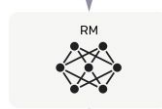


The policy
generates
an output.

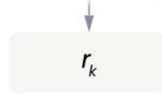


Once upon a time...

The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



chatGPT (instructGPT):

LLM base

Predice siguiente token en base a los datos de entrenamiento

Once upon a time there was a boy that wanted to become a king.

What is the capital of Chile?

What is the population of Chile?

What is the main economic activity of Chile?

...

chatGPT (instructGPT):

LLM instruccional

Intenta responder a las solicitudes del usuario

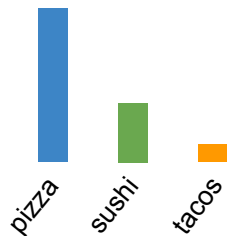
What is the capital of Chile?

The capital of Chile is Santiago

chatGPT:

Temperatura

Cuál es tu comida favorita?



Temperatura = 0.0

Cuál es tu comida favorita? **Pizza**
Cuál es tu comida favorita? **Pizza**
Cuál es tu comida favorita? **Pizza**

Temperatura = 0.3

Cuál es tu comida favorita? **Pizza**
Cuál es tu comida favorita? **Sushi**
Cuál es tu comida favorita? **Pizza**

Temperatura = 0.7

Cuál es tu comida favorita? **Tacos**
Cuál es tu comida favorita? **Pizza**
Cuál es tu comida favorita? **Sushi**

chatGPT:

```
messages =  
[  
  {"role": "system",  
   "content": "You are an assistant... "},  
  {"role": "user",  
   "content": "tell me a joke "},  
  {"role": "assistant",  
   "content": "Why did the chicken... "},  
  ...  
]
```

```
messages =  
[  
  system  
  user  
  assistant  
  user  
  assistant  
  ...  
]
```

Contenidos de la clase de hoy

- 1** **Introducción**
¿Por qué tantos modelos?
- 2** **ELMo**
Modelo
Aplicaciones
- 3** **BERT**
Modelo
Aplicaciones
- 4** **GPT**
Modelos GPT 1, 2 y 3, chatGPT y GPT-4
Aplicaciones
- 5** **Herramientas Concretas**
Introducción
Trabajo práctico

Herramientas

**HUGGING FACE**

[Back to home](#)

All Models and checkpoints

Also check out our list of [Community contributors](#) and [Organizations](#).

Tags: pytorch ▾ Sort: Most downloads ▾

[deepset/bert-large-uncased-whole-word-masking-squad2](#)

[bert-base-uncased](#) ★

[distilbert-base-uncased](#) ★

[deepset/roberta-base-squad2](#) ★

**transformers**
v3.3.0 ▾
Star 34,686

Docs ▸ Training and fine-tuning

[Open in Colab](#) [View page source](#)

Training and fine-tuning

Model classes in  Transformers are designed to be compatible with native PyTorch and TensorFlow 2 and can be used seamlessly with either. In this quickstart, we will show how to fine-tune (or train from scratch) a model using the standard training tools available in either framework. We will also show how to use our included `Trainer()` class which handles much of the complexity of training for you.

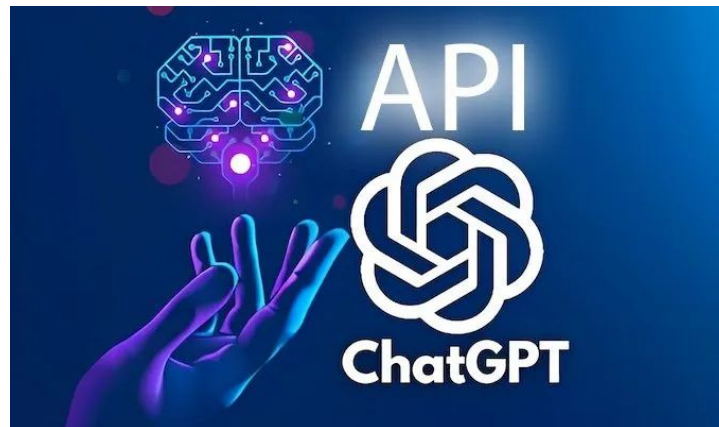
This guide assumes that you are already familiar with loading and using our models for inference; otherwise, see the [task summary](#). We also assume that you are familiar with training deep neural networks in either PyTorch or TF2, and focus specifically on the nuances and tools for training models in  Transformers.

Sections:

- Fine-tuning in native PyTorch
- Fine-tuning in native TensorFlow 2
- Trainer
- Additional resources

Fine-tuning in native PyTorch

Model classes in  Transformers that don't begin with `Tr` are **PyTorch Modules**, meaning that you can use them just as you would any model in PyTorch for both inference and optimization.





Pontificia Universidad Católica de Chile
Departamento de Ciencias de la Computación

Diplomado en Inteligencia Artificial

Clase 20

ELMo, BERT, GPT, ChatGPT

Carlos Aspillaga



IA Lab

CENIA
CENTRO NACIONAL DE INTELIGENCIA ARTIFICIAL